



1495

UNIVERSITY OF
ABERDEEN

CELEBRATING
525 YEARS
1495 – 2020

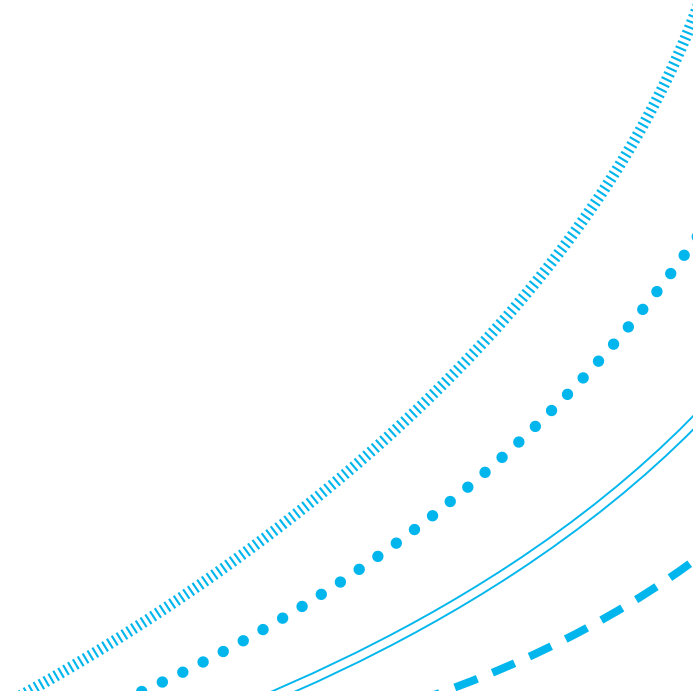
ABERDEEN 2040

Naïve Bayes

Data Mining & Visualisation
Lecture 13

Today...

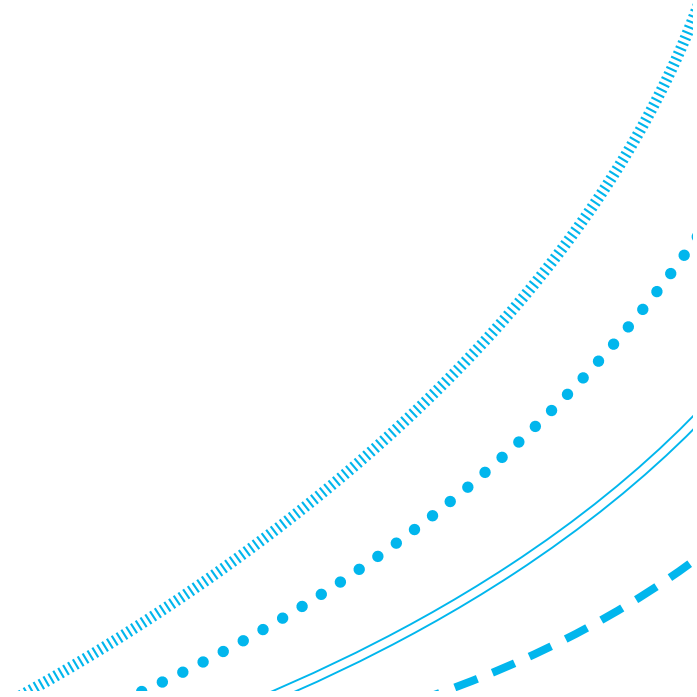
- Generative vs Discriminative Classifiers
- Bayes' Theorem
- Naïve Bayes



Today...

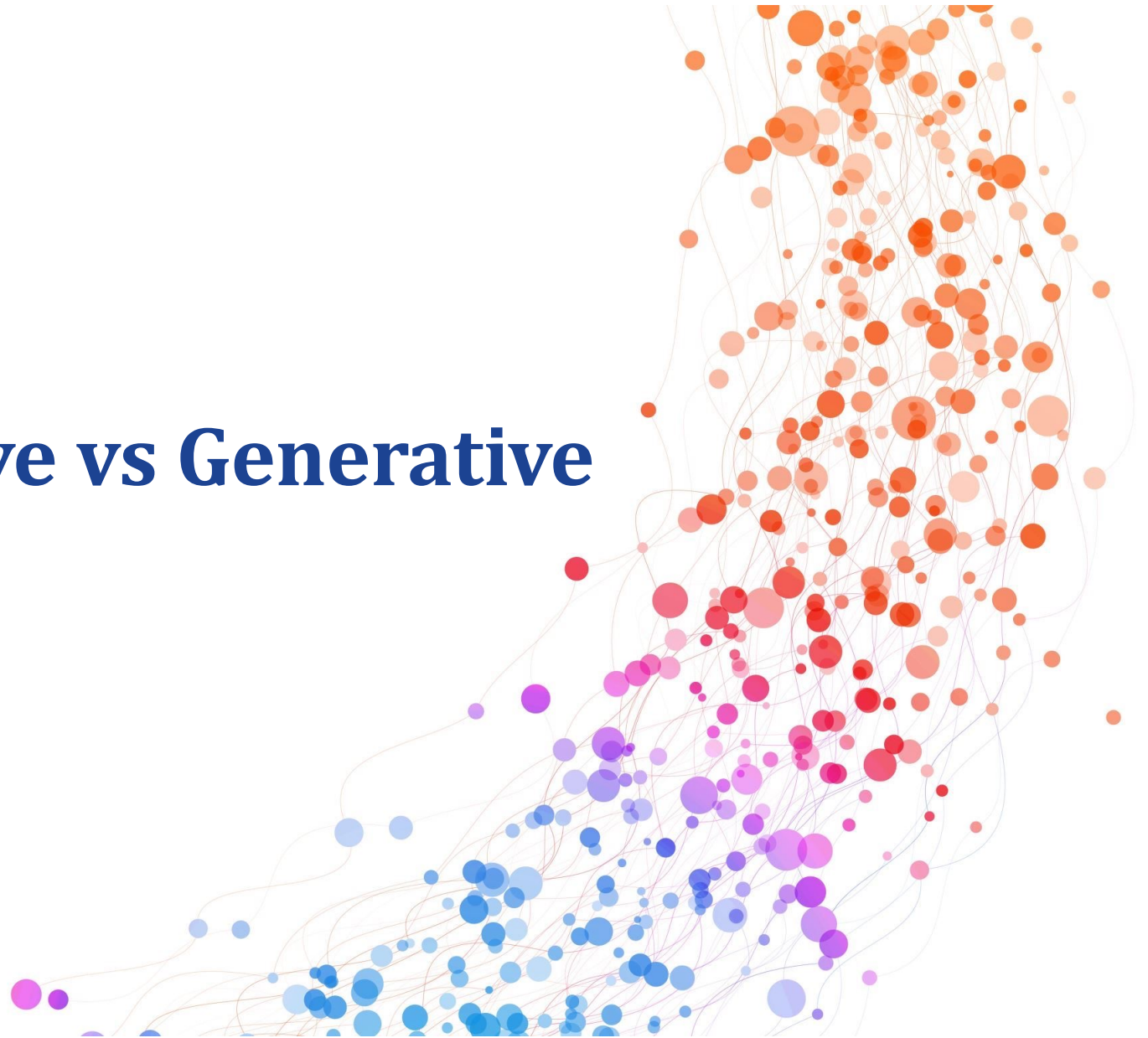
More broadly, we're going to continue our discussion on classification, focusing on two popular approaches:

- Naïve Bayes Classifiers
- Support Vector Machines



Discriminative vs Generative Classifiers

ABERDEEN 2040



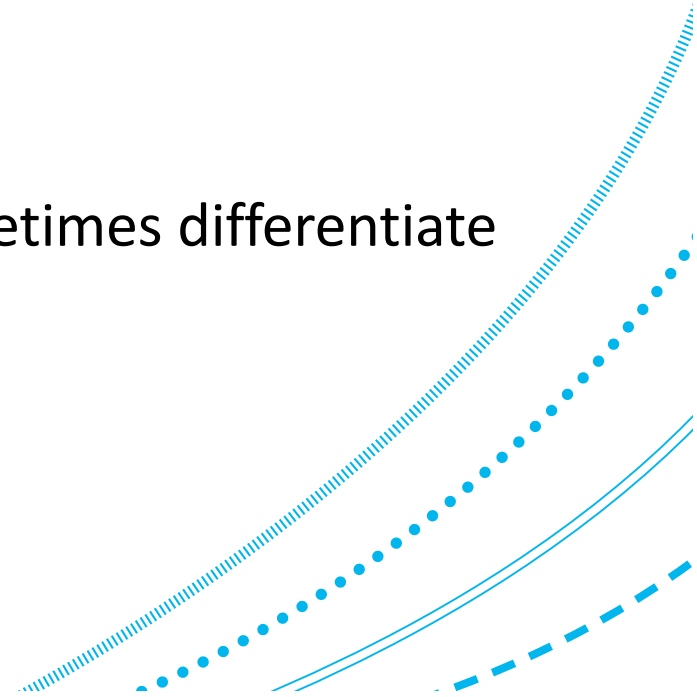
Discriminative vs Generative Classifiers

When we're looking to classify some categorical variable (Y), using some observable variable(s) (X), we look to estimate:

$$P(Y \mid X)$$

And when dealing with classification tasks, we sometimes differentiate between two main approaches:

- **Discriminative classifiers**
- **Generative classifiers**

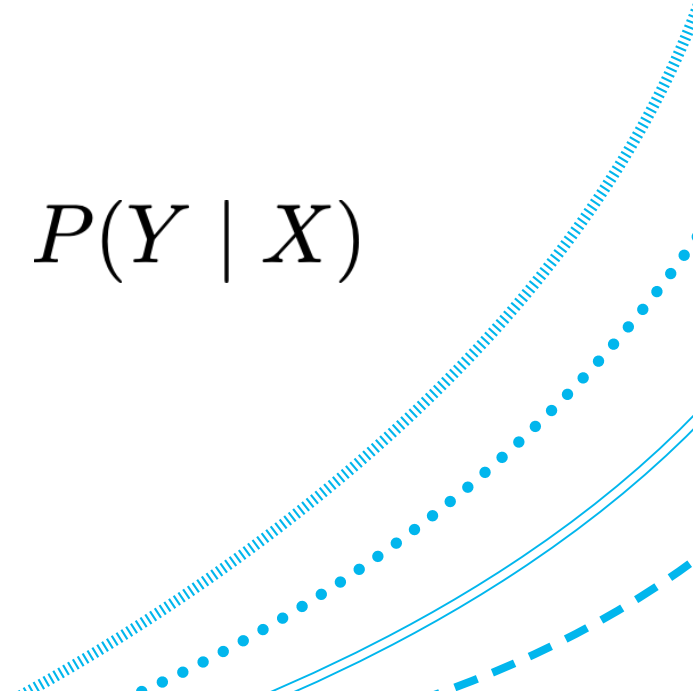


Discriminative Classifiers

Discriminative Classifiers look to model the conditional probability of the target Y , given an observable x :

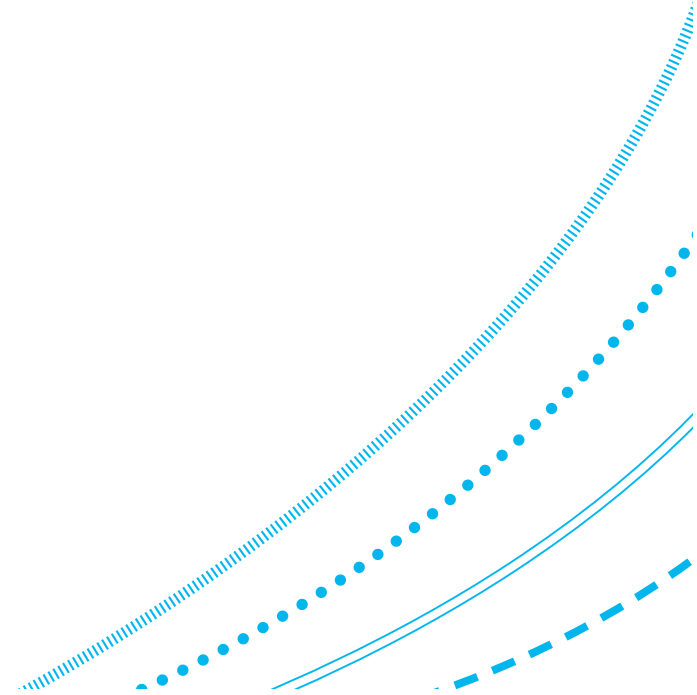
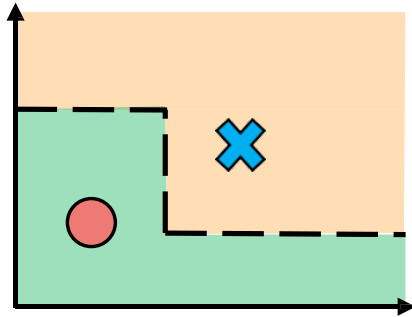
$$P(Y \mid X = x)$$

They do this through estimating the parameters of $P(Y \mid X)$ directly through the training data.



Discriminative Classifiers

Logistic regression and decision trees are both examples of discriminative classifiers, given that they use the training data to distinguish decision boundaries between classes.



Generative Classifiers

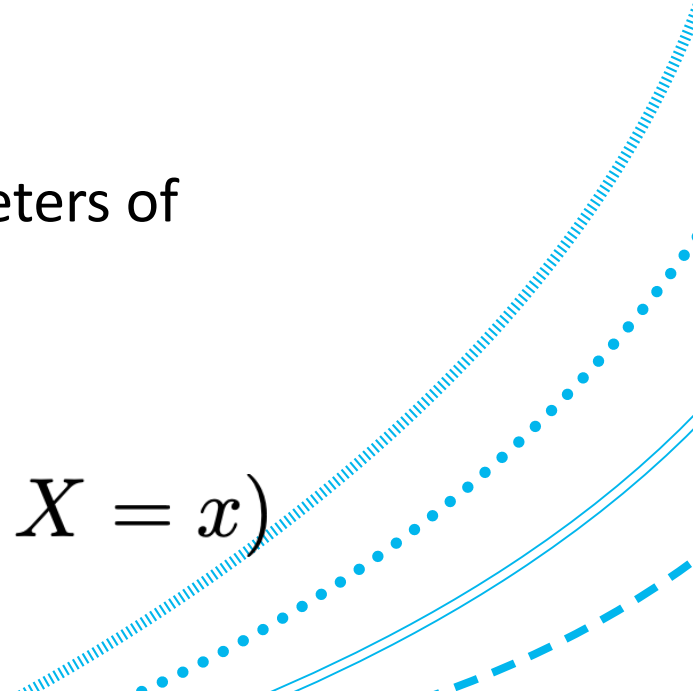
In contrast, **Generative Classifiers** look to model the conditional probability of the observable X , given a target y :

$$P(X \mid Y = y)$$

They do this through instead estimating the parameters of

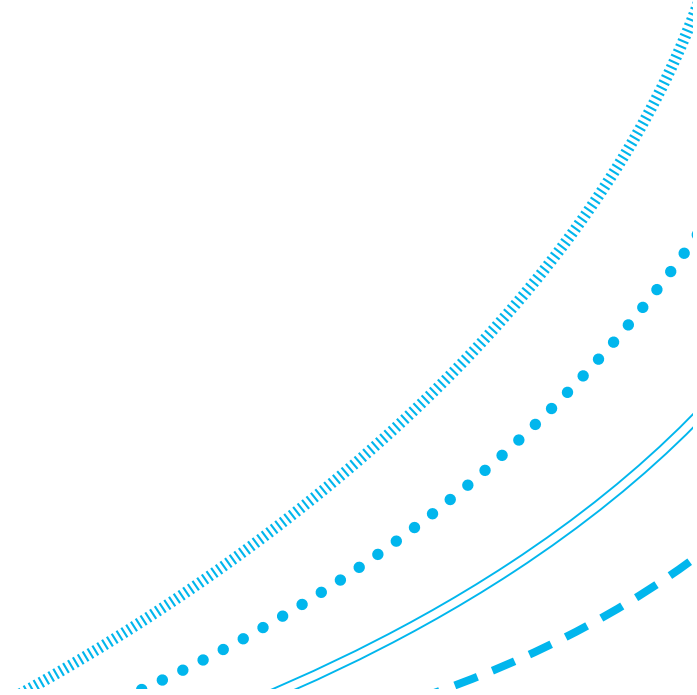
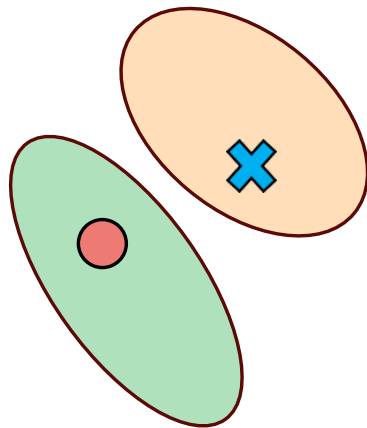
$$P(X \mid Y), P(Y)$$

and then use **Bayes' Theorem** to calculate $P(Y \mid X = x)$



Generative Classifiers

Two of the most well-known generative classifiers are the Naïve Bayes Classifier (the subject of this lecture), and Linear Discriminant Analysis (LDA) – which we'll get to another day...



Bayes' Theorem

ABERDEEN 2040

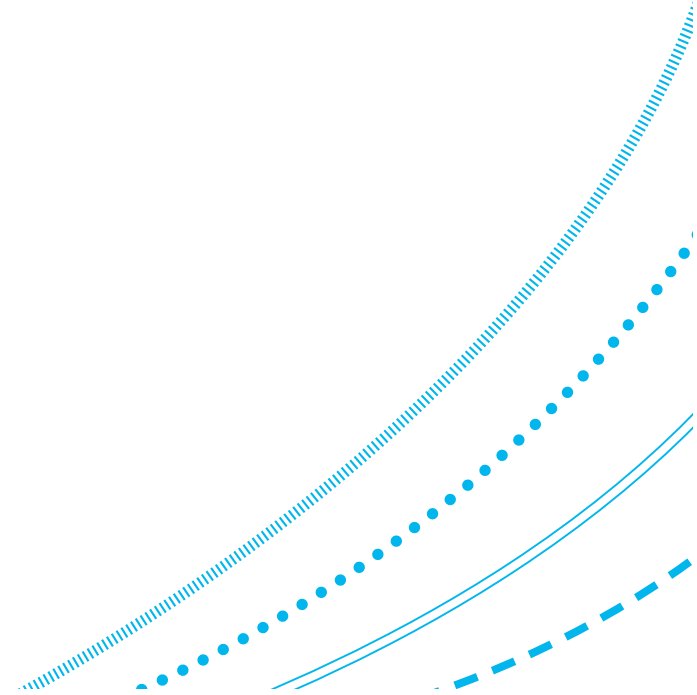


Bayes' Theorem

Thomas Bayes was a statistician and philosopher who lived in the 1700s.

Today, he is most well-known for his theorem:

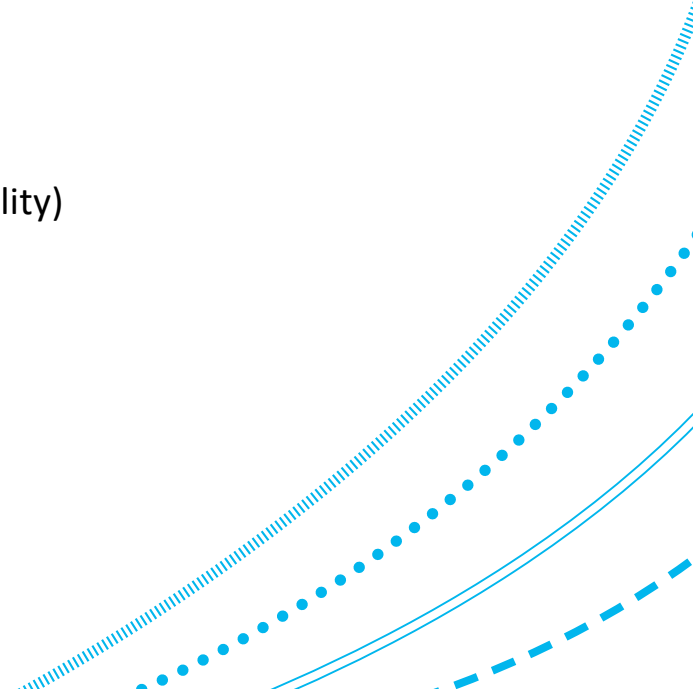
$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$



Bayes' Theorem

Bayes' theorem is used to determine the probability of a hypothesis, based on prior knowledge of conditions that are assumed to be relevant to that hypothesis.

$$\begin{array}{c} \text{(Posterior probability)} \\ P(A|B) = \frac{\begin{array}{cc} \text{(Likelihood)} & \text{(Prior probability)} \\ P(B|A)P(A) \end{array}}{\begin{array}{c} P(B) \\ \text{(Evidence)} \end{array}} \end{array}$$



Bayes' Theorem: Example

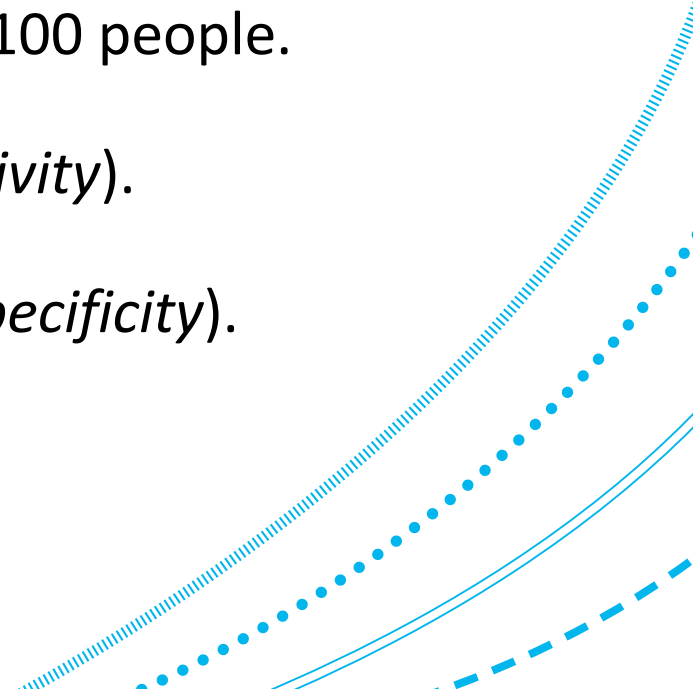
Let's say you go to the doctor with some unusual symptoms, and the doctor runs a few tests.

You test positive for a rare condition that affects 1/100 people.

99% of people with the disease test positive (*sensitivity*).

94% of people without the disease test negative (*specificity*).

How worried should you be?



Bayes' Theorem: Example

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

$$P(\text{Have disease}) = 0.01$$

$$\text{Therefore, } P(\text{Do not have disease}) = 0.99$$

$$P(\text{Positive test} | \text{Have disease}) = 0.99$$

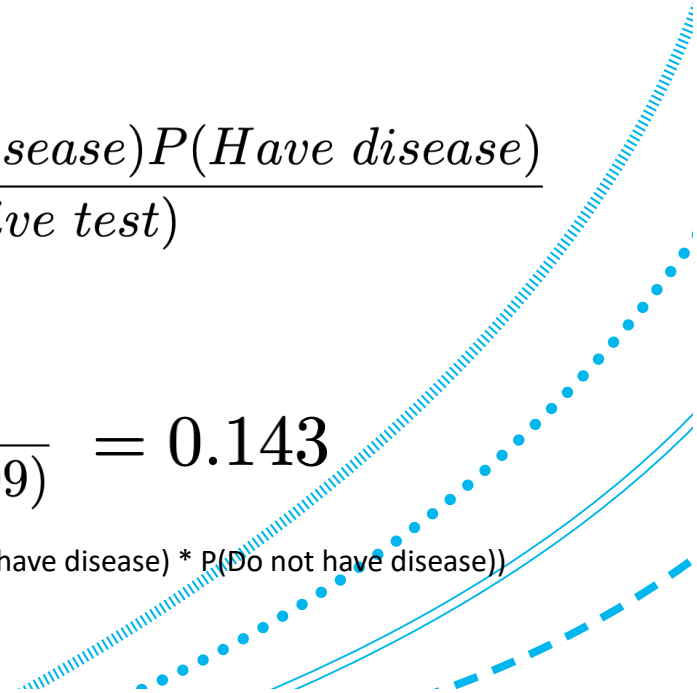
$$P(\text{Negative test} | \text{Do not have disease}) = 0.94$$

$$\text{Therefore, } P(\text{Positive test} | \text{Do not have disease}) = 0.06$$

$$P(\text{Have disease} | \text{Positive test}) = \frac{P(\text{Positive test} | \text{Have disease})P(\text{Have disease})}{P(\text{Positive test})}$$

$$P(\text{Have disease} | \text{Positive test}) = \frac{0.99 * 0.01}{(0.99 * 0.01) + (.06 * 0.99)} = 0.143$$

Note that $P(\text{Positive test}) = (P(\text{Positive test} | \text{Have disease}) * P(\text{Have disease})) + (P(\text{Positive test} | \text{Do not have disease}) * P(\text{Do not have disease}))$

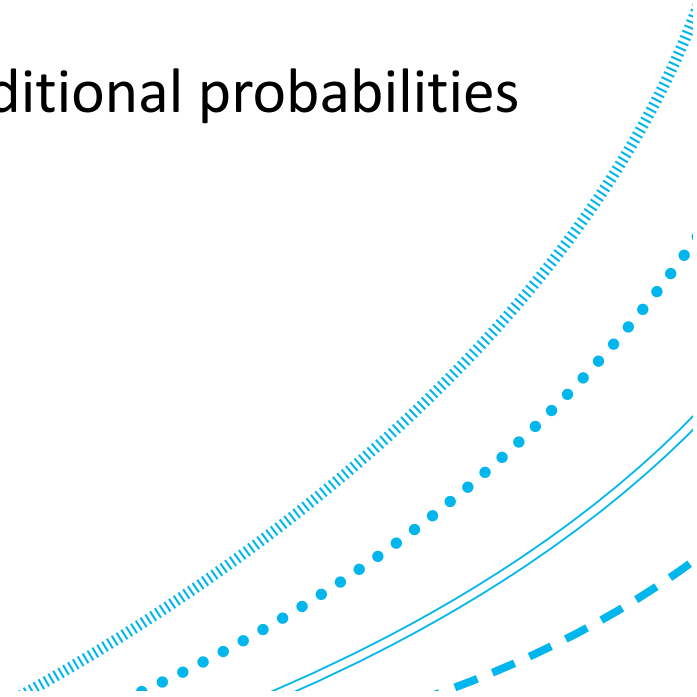


Bayes' Theorem

So even with a positive test, there's still an 85.7% chance that you don't actually have the disease.

In short, Bayes' Theorem allows us to calculate conditional probabilities based on prior knowledge.

So let's see how this can apply to classification!



Naïve Bayes Classification

ABERDEEN 2040

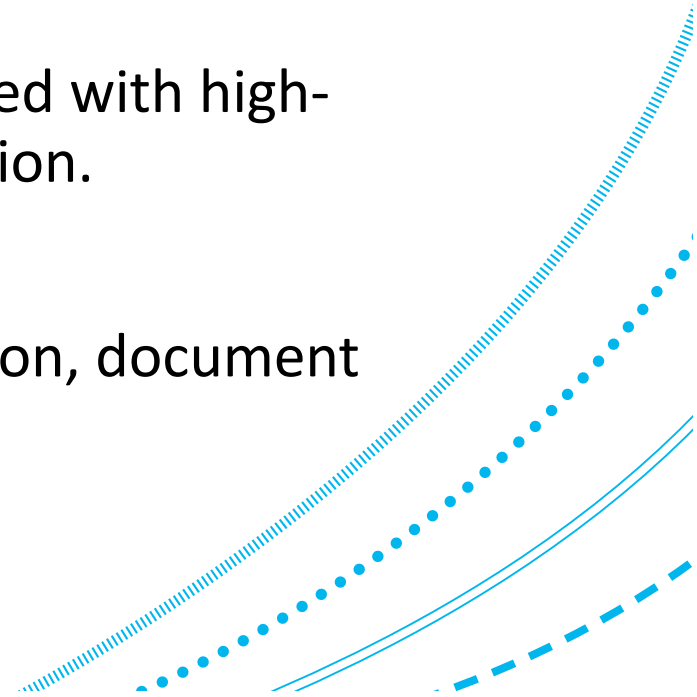


Naïve Bayes

Naïve Bayes is the name given to a family of supervised learning algorithms for classification.

It is *based* on **Bayes' Theorem**, and is frequently used with high-dimensional datasets, such as within text classification.

Common examples include email (spam) classification, document classification, sentiment analysis, etc.



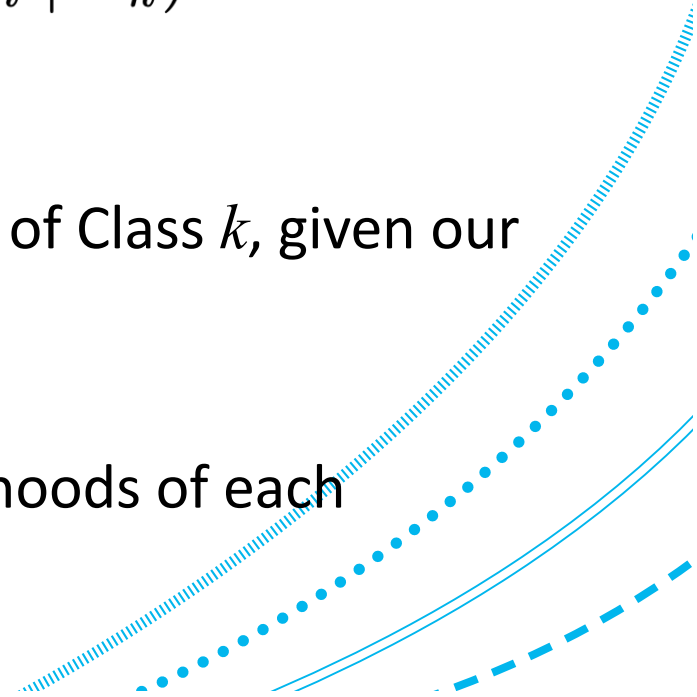
Naïve Bayes Formula

When we're doing naïve Bayes classification, we use:

$$P(C_k | x_1, \dots, x_n) \propto P(C_k) \prod_{i=1}^n P(x_i | C_k)$$

Where:

- $P(C_k | x_1, \dots, x_n)$ is the posterior probability of Class k , given our features.
- $P(C_k)$ is the prior probability of Class k .
- $\prod_{i=1}^n P(x_i | C_k)$ is the product of the likelihoods of each feature, given the class.



Naïve Bayes Formula

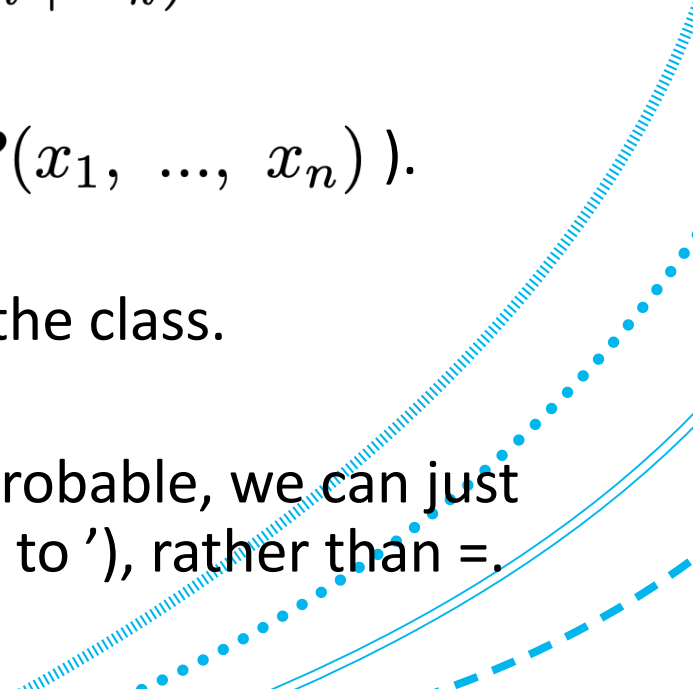
When we're doing naïve Bayes classification, we use:

$$P(C_k | x_1, \dots, x_n) \propto P(C_k) \prod_{i=1}^n P(x_i | C_k)$$

Note that we don't include the denominator (i.e. $P(x_1, \dots, x_n)$).

This is because this value is the same regardless of the class.

Given that we're interested in which class is more probable, we can just ignore it. This is also why we use $\propto$ ('is proportional to'), rather than $=$.



Class Conditional Independence Assumption

Naïve Bayes classifiers are naïve in the sense that they assume **all features are conditionally independent, given the class**.

For example, in spam classification, within each class, we assume that all words are independent of each other.

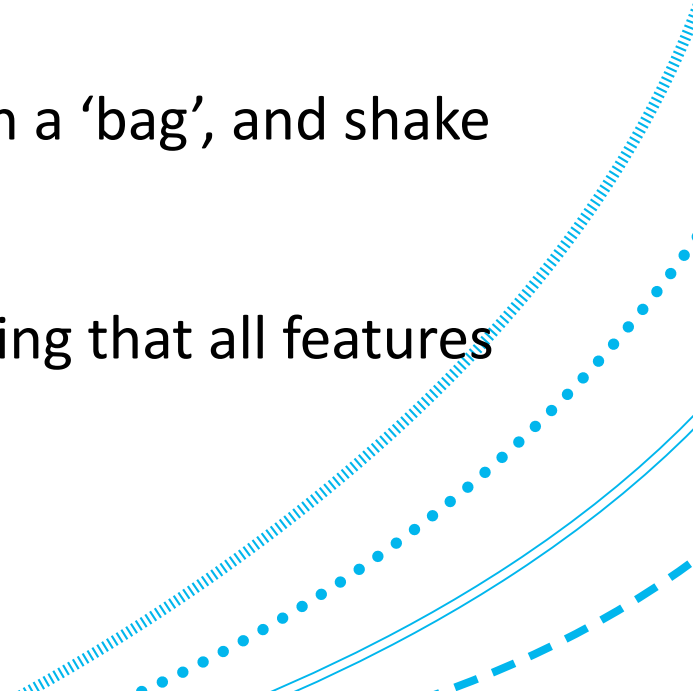


The 'Bag of Words' Model

When talking about the **Class Conditional Independence Assumption** within the context of text analysis, we often use the phrase 'bag of words'.

Imagine we take every word in a document, put it in a 'bag', and shake that bag up. We discard all ordering, context, etc.

This is essentially what it means when we're assuming that all features are independent of each other.

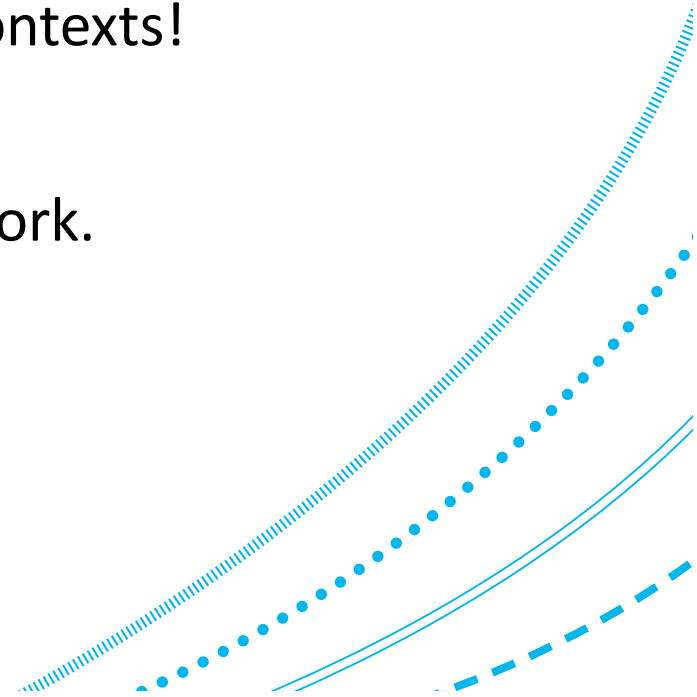


How Good are Naïve Bayes Classifiers?

So despite their naivety, how good are Naïve Bayes Classifiers?

In practice, they actually work fairly well in many contexts!

Let's walk through an example to show how they work.

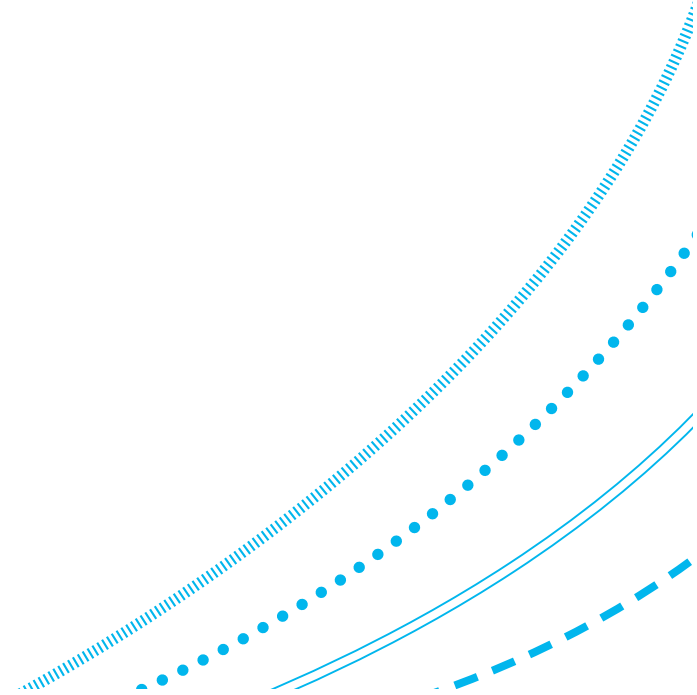


Example: Spam Email Classification

Let's say we want to build an email spam filter.

We have a dataset of emails:

- 75% are 'Normal' emails
- 25% are 'Spam' emails



Example: Spam Email Classification

We can start by counting the number of instances of each word in both the Normal subset and the Spam subset.

Normal (75%)

Hey: 8

Uni: 6

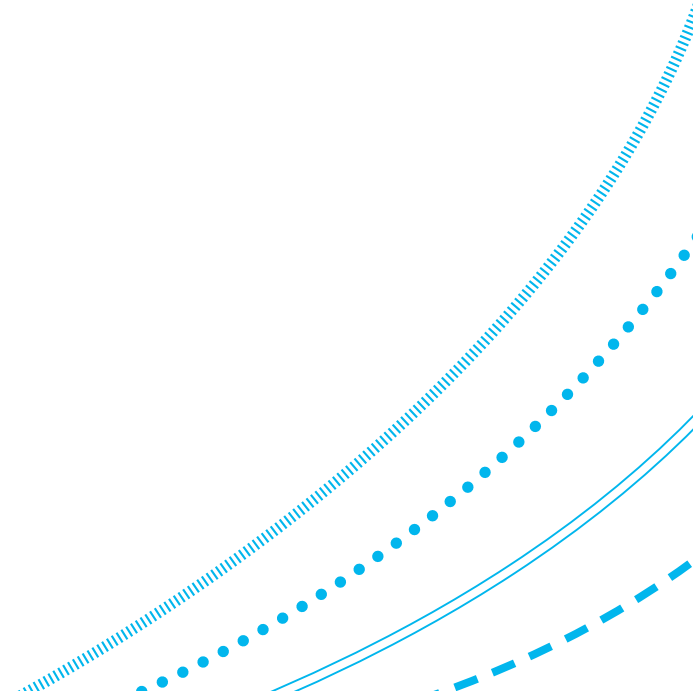
Bank: 2

Spam (25%)

Hey: 2

Uni: 1

Bank: 6



Example: Spam Email Classification

We then calculate the probabilities for each word in each class.

$$P(\mathbf{N}) = .75$$

$$P(\text{Hey}|\mathbf{N}) = 8 / 16 = .5$$

$$P(\text{Uni}|\mathbf{N}) = 6 / 16 = .375$$

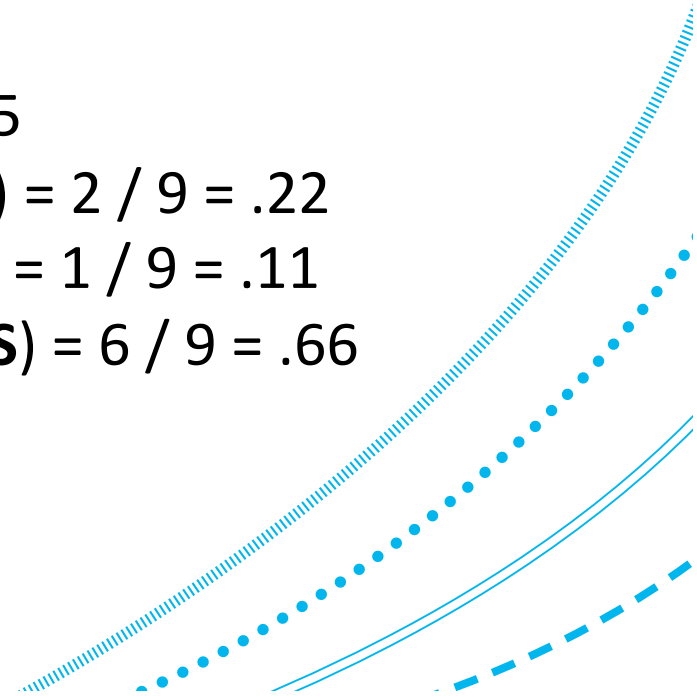
$$P(\text{Bank}|\mathbf{N}) = 2 / 16 = .125$$

$$P(\mathbf{S}) = .25$$

$$P(\text{Hey}|\mathbf{S}) = 2 / 9 = .22$$

$$P(\text{Uni}|\mathbf{S}) = 1 / 9 = .11$$

$$P(\text{Bank}|\mathbf{S}) = 6 / 9 = .66$$



Example: Spam Email Classification

$$P(\mathbf{N}) = .75$$

$$P(\text{Hey}|\mathbf{N}) = 8 / 16 = .5$$

$$P(\text{Uni}|\mathbf{N}) = 6 / 16 = .375$$

$$P(\text{Bank}|\mathbf{N}) = 2 / 16 = .125$$

$$P(\mathbf{S}) = .25$$

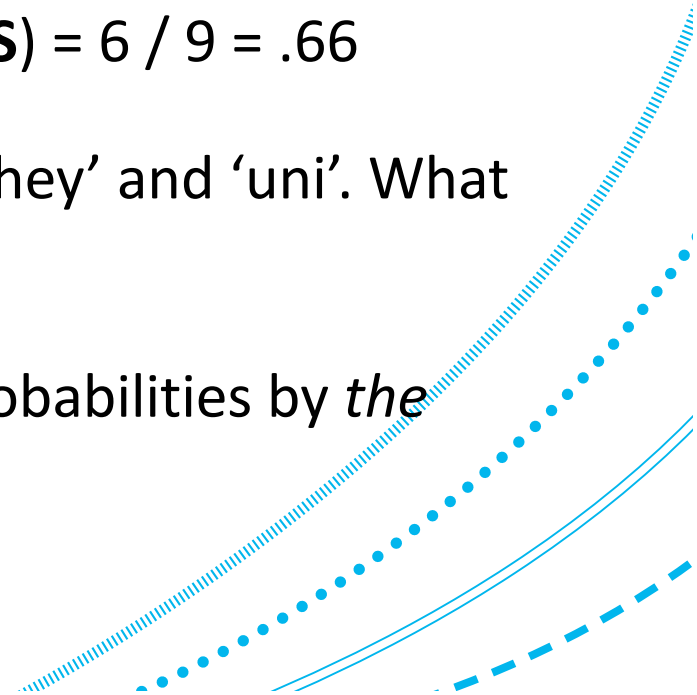
$$P(\text{Hey}|\mathbf{S}) = 2 / 9 = .22$$

$$P(\text{Uni}|\mathbf{S}) = 1 / 9 = .11$$

$$P(\text{Bank}|\mathbf{S}) = 6 / 9 = .66$$

We receive a new email which contains the words 'hey' and 'uni'. What is the probability that it's spam?

For each class, we multiply each of these words' probabilities by *the prior probability* of that class occurring.



Example: Spam Email Classification

$$P(\mathbf{N}) = .75$$

$$P(\text{Hey}|\mathbf{N}) = 8 / 16 = .5$$

$$P(\text{Uni}|\mathbf{N}) = 6 / 16 = .375$$

$$P(\text{Bank}|\mathbf{N}) = 2 / 16 = .125$$

$$P(\mathbf{S}) = .25$$

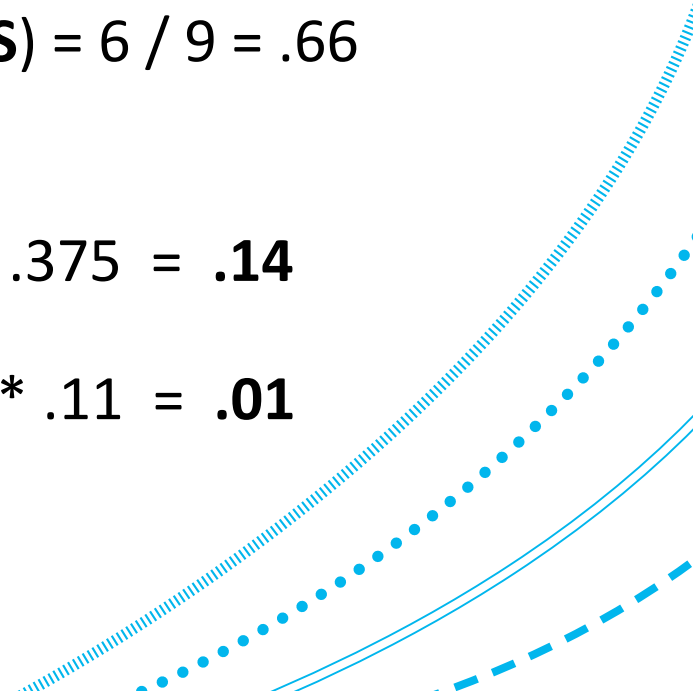
$$P(\text{Hey}|\mathbf{S}) = 2 / 9 = .22$$

$$P(\text{Uni}|\mathbf{S}) = 1 / 9 = .11$$

$$P(\text{Bank}|\mathbf{S}) = 6 / 9 = .66$$

$$\text{Normal: } P(\mathbf{N}) * P(\text{Hey}|\mathbf{N}) * P(\text{Uni}|\mathbf{N}) = .75 * .5 * .375 = \mathbf{.14}$$

$$\text{Spam: } P(\mathbf{S}) * P(\text{Hey}|\mathbf{S}) * P(\text{Uni}|\mathbf{S}) = .25 * .22 * .11 = \mathbf{.01}$$



Example: Spam Email Classification

$$P(\mathbf{N}) = .75$$

$$P(\text{Hey}|\mathbf{N}) = 8 / 16 = .5$$

$$P(\text{Uni}|\mathbf{N}) = 6 / 16 = .375$$

$$P(\text{Bank}|\mathbf{N}) = 2 / 16 = .125$$

$$P(\mathbf{S}) = .25$$

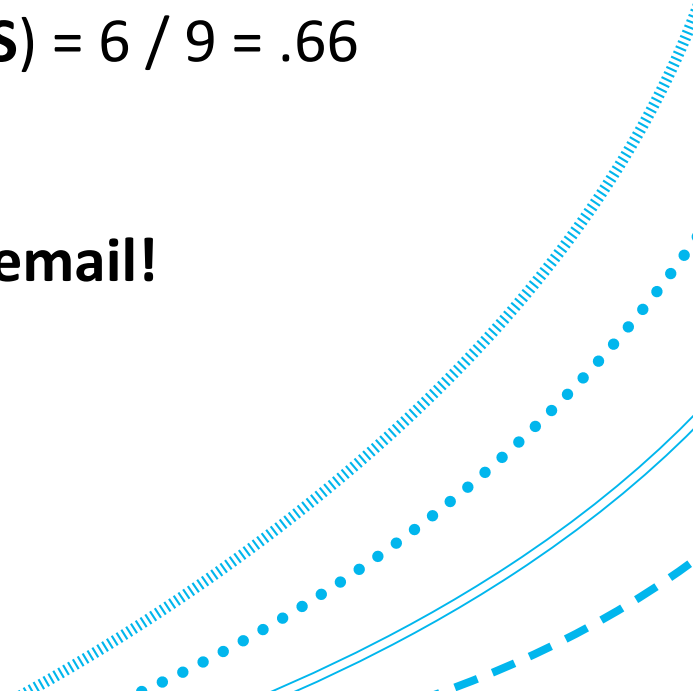
$$P(\text{Hey}|\mathbf{S}) = 2 / 9 = .22$$

$$P(\text{Uni}|\mathbf{S}) = 1 / 9 = .11$$

$$P(\text{Bank}|\mathbf{S}) = 6 / 9 = .66$$

$P(\mathbf{N}|\text{Hey, Uni}) \propto .14$ **So we assume it's a normal email!**

$P(\mathbf{S}|\text{Hey, Uni}) \propto .01$



Naïve Bayes and the Zero Probability Problem

ABERDEEN 2040



The Zero Probability Problem

$$P(\mathbf{N}) = .75$$

$$P(\text{Hey}|\mathbf{N}) = 8 / 16 = .5$$

$$P(\text{Uni}|\mathbf{N}) = 6 / 16 = .375$$

$$P(\text{Bank}|\mathbf{N}) = 2 / 16 = .125$$

$$P(\text{Friend}|\mathbf{N}) = 0 / 16 = 0$$

$$P(\mathbf{S}) = .25$$

$$P(\text{Hey}|\mathbf{S}) = 2 / 9 = .22$$

$$P(\text{Uni}|\mathbf{S}) = 0 / 9 = 0$$

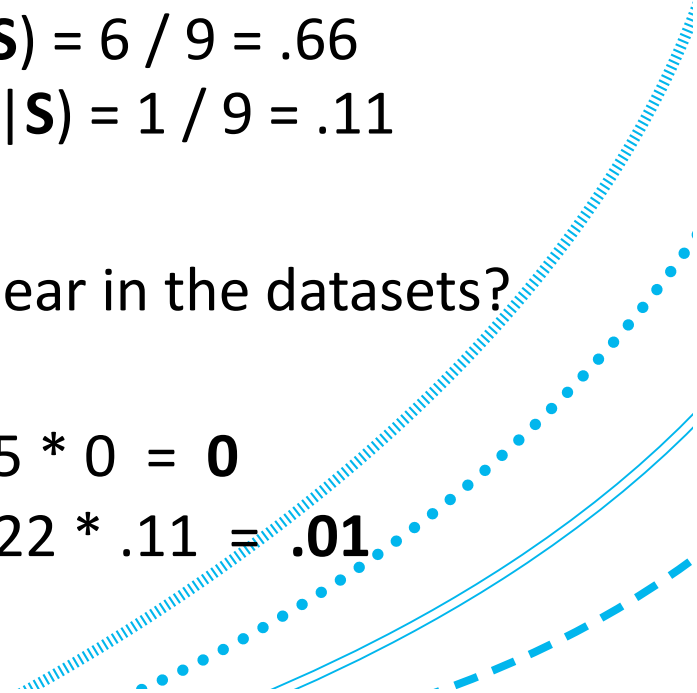
$$P(\text{Bank}|\mathbf{S}) = 6 / 9 = .66$$

$$P(\text{Friend}|\mathbf{S}) = 1 / 9 = .11$$

But what happens if there are words that don't appear in the datasets?

$$\text{Normal: } P(\mathbf{N}) * P(\text{Hey}|\mathbf{N}) * P(\text{Friend}|\mathbf{N}) = .75 * .5 * 0 = \mathbf{0}$$

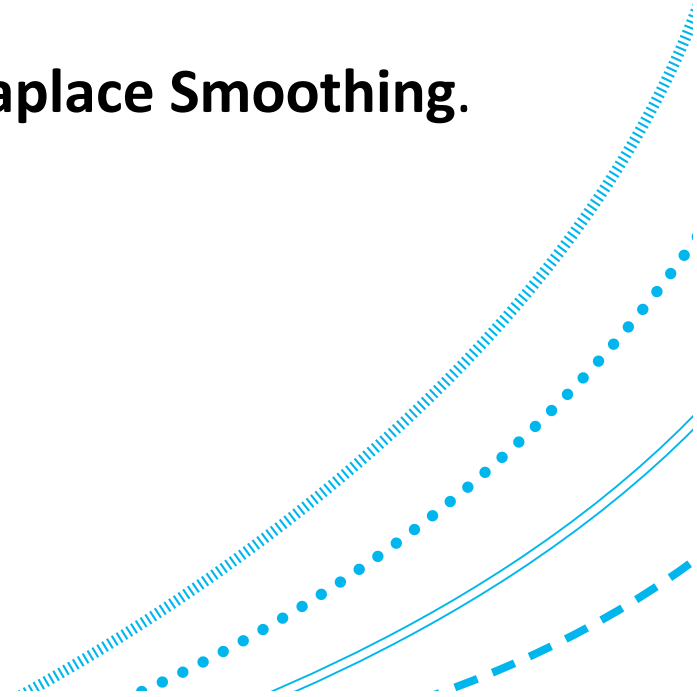
$$\text{Spam: } P(\mathbf{S}) * P(\text{Hey}|\mathbf{S}) * P(\text{Friend}|\mathbf{S}) = .25 * .22 * .11 = \mathbf{.01}$$



The Zero Probability Problem

In such cases, the new word will always result in the posterior probability becoming 0 regardless of the likelihoods of other words.

To avoid this, we can apply a technique known as **Laplace Smoothing**.

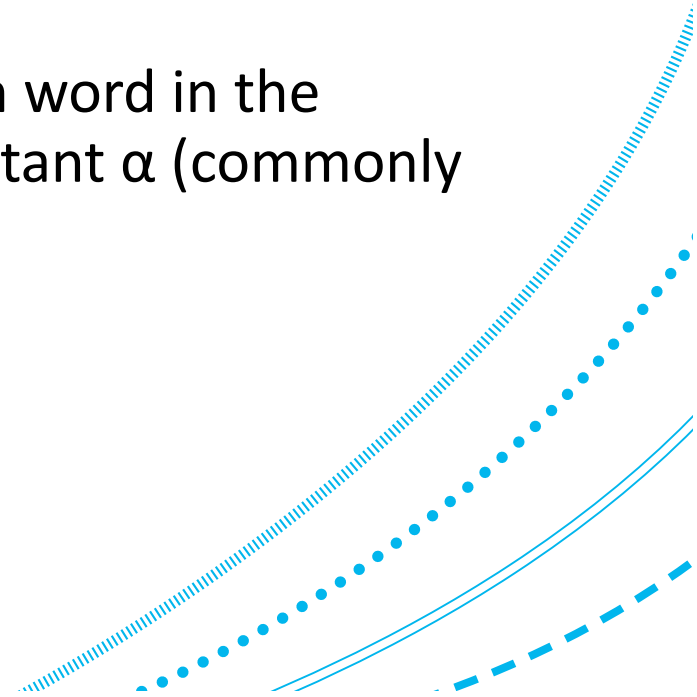


Laplace Smoothing

With Laplace Smoothing, we calculate probabilities in the same way as we did before.

When we're counting the number instances of each word in the training set for each class, we now add a small constant α (commonly 1) to each word.

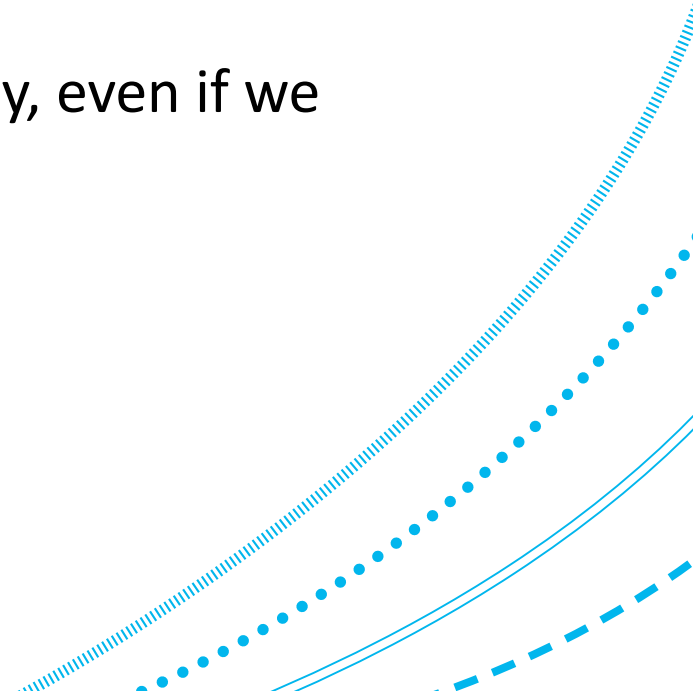
$$P(x_i \mid C_k) = \frac{\text{count}(x_i, C_k) + \alpha}{\text{total words in class } C_k}$$



Laplace Smoothing

When we come across a new word, we just set the count of that word to α .

As a result, all words will have a non-zero probability, even if we haven't seen them before.



Strengths and Limitations of Naïve Bayes Classification

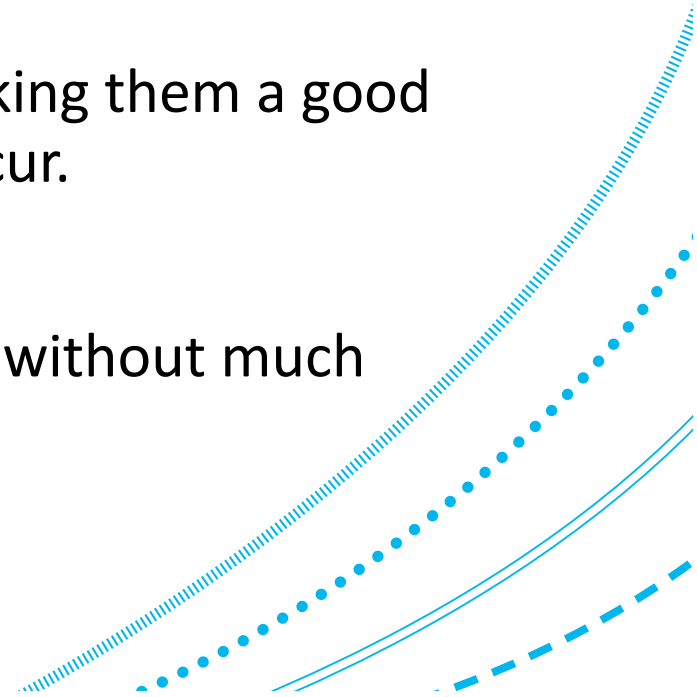


Strengths and Limitations

Naïve Bayes classifiers typically excel on tasks with high-dimensional data (such as spam detection, document classification, etc.).

They are fast, efficient, and fairly interpretable, making them a good choice for understanding why certain outcomes occur.

They can also be used on multi-class (3+) problems without much added complexity.



Strengths and Limitations

However, the class conditional independence assumption is often unrealistic. We lose valuable contextual information by discarding the ordering of words.

As a result, more complex approaches will often outperform naïve Bayes, if interpretability and resource constraints are not an issue.

